InboxPilot: An Agentic AI Application

Architecting Foundational Models into Autonomous Systems

Drishti IAS

March 17, 2026

Agenda

1. 1. Introduction to InboxPilot
2. 2. Development Artefacts
3. 3. System Architecture & LangGraph Diagram
4. 4. Conclusion

What is InboxPilot?

Overview

InboxPilot is a specialized Agentic AI application designed to autonomously process, classify, and respond to incoming email and text streams using a distributed multi-agent system.

What It Does

- **Ingests:** Reads continuous data streams via tools (e.g., IMAP).
- **Reasons:** Dynamically synthesizes context.
- **Acts:** Calls external APIs to retrieve specific missing data.

The Agents Involved

- **Classify Agent:** Detects user intent.
- **Fetch Agent:** Searches and retrieves context.
- **Respond Agent:** Generates final drafts.
- **Supervisor:** Orchestrates routing logic.

The Core Artefacts

Our framework divides the ecosystem into four fundamental pillars, essential for any robust Agentic platform:

1. **Models:** Foundational reasoning engines and schemas.
2. **Tools:** Actionable capabilities connecting to the outside world.
3. **Agents:** Specialized autonomous workers maintaining context.
4. **Orchestration:** Directed logic governing overarching data flow.

Artefact 1: Models

Models form the brain of the platform. We rely on standard abstractions loaded directly into the execution environment.

Source Components

- `config.yaml`: Static definitions for model hyperparameters.
- `inboxpilot/models.py`: Pydantic structures acting as exact schemas for validation.
- `inboxpilot/state.py`: The memory structure passed continuously during runtime.

Runtime: Loaded natively into the Python environment and injected as configuration context across all executing containers.

Artefact 2: Tools

Language models alone cannot act. *Tools* provide the APIs to enact change in the external world.

Application Tools (`inboxpilot/tools/`)

- `calendar_tool.py`: External calendar management.
- `imap_tool.py`: Email stream ingestion.
- `search_tool.py`: Open-world information retrieval.

Runtime Integration: Executed as live, parameter-driven functions operating securely within an Agent's isolated execution process.

Artefact 3: Agents

Agents are independent micro-nodes of intelligence. Each agent in InboxPilot is bounded to a specific persona and logic flow.

The Workforce

- **Classify Agent:** Determines user intent from the raw input payload.
- **Fetch Agent:** Retrieves required context via dynamic tool usage.
- **Respond Agent:** Formulates the final synthesized output.

Microservices: Deployed as live ASGI microservices. Each agent has its own `Dockerfile` (e.g., `services/classify-service/Dockerfile`) to guarantee clean, scalable bounds.

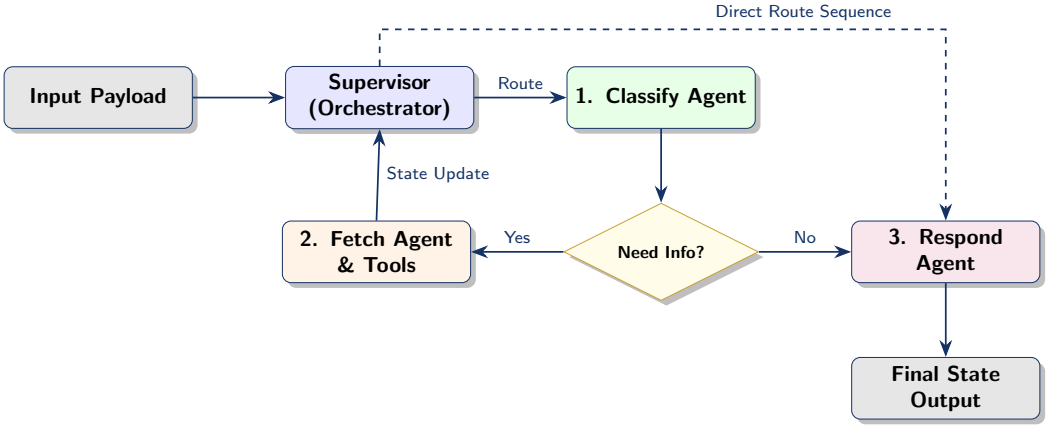
Artefact 4: Orchestration

If Agents are the workers, Orchestration is the manager. It handles complex interactions across the distributed system.

Components

- **Supervisor Node (`services/supervisor/`):** Houses `graph.py`, dynamically routing logic between the workforce.
- **Communication (`a2a/`):** Agent-to-Agent message passing protocol over the internal network.
- **Infrastructure (`docker-compose.yml`):** Links the master orchestrator to its subordinate agents.

System Architecture & DAG Routing



Dynamic LangGraph Execution: The Orchestrator routes data and maintains the overarching state memory.

Conclusion & Summary of Achievements

- **Separation of Concerns:** Rigidly partitioned Models, Tools, Agents, and Orchestration logic.
- **Scalability:** Containerized components into a distributed microservice web via Docker.
- **Autonomy:** Established an event-driven reasoning system capable of dynamic routing and state persistence.

Looking Forward

Establishing **InboxPilot** as a foundational reference architecture for future enterprise-grade, multi-agent platforms.

Thank You

Questions?